

Authentication Frontend

Séance 9 - Se connecter depuis React : login, token, contexte

Cours 420-941-MA - Collège de Maisonneuve - Groupe 25604

login - localStorage - header Authorization - AuthContext - routes protégées - logout

Où on en est, où on va

Récap S6-S8 et objectifs S9

Acquis

- Backend : JWT + rôles (S6)
- Front React : hooks, axios (S7-S8)
- Le Pokedex front consomme l'API
- On sait faire /auth/login (.rest)
- MAIS : le front ne se connecte pas encore

Objectifs de la séance

- Un formulaire de connexion en React
- Récupérer et STOCKER le token
- L'envoyer dans le header Authorization
- AuthContext : état global 'connecté'
- Afficher/masquer selon la connexion
- Routes protégées + déconnexion

La dernière pièce. Le backend sait déjà QUI a le droit (JWT + rôles, séance 6). Côté front, on affichait des données publiques. Aujourd'hui, on ferme la boucle : l'utilisateur se CONNECTE dans l'interface, l'app garde son token, et l'envoie à chaque requête protégée. C'est le vrai flux d'une application moderne.

PARTIE 1

Rappel : le cycle JWT (front <-> back)

Le cycle d'authentification

Ce que le front doit orchestrer



Côté backend (déjà fait, S6)

- POST /auth/login verifie le mot
- de passe (bcrypt)
- Renvoie un token signe (JWT)
- Les routes protegees exigent
- le header Authorization

Côté frontend (aujourd'hui)

- Formulaire de connexion
- Appel `axios.post('/auth/login')`
- Stocker le token reçu
- L'ajouter a chaque requete
- Gerer l'etat connecte / deconnecte

PARTIE 2

Login, token et requêtes

Le formulaire de connexion

Un formulaire contrôlé + axios.post

```
function Login() {
  const [email, setEmail] = useState('')
  const [mdp, setMdp] = useState('')
  const [erreur, setErreur] = useState('')

  async function seConnecter(e: React.FormEvent) {
    e.preventDefault()
    try {
      const { data } = await axios.post(
        'http://localhost:3000/auth/login',
        { email, password: mdp })
      localStorage.setItem('token', data.token) // on garde le token
    } catch {
      setErreur('Identifiants invalides')
    }
  }
  // ... <form onSubmit={seConnecter}> (2 inputs contrôlés)
}
```

TSX

Les points clés

- Formulaire contrôlé
- (value + onChange)
-
- e.preventDefault()
-
- axios.post envoi
- email + password
-
- on REÇOIT data.token
- et on le stocke
-
- on gère l'erreur (401)

Où stocker le token ?

localStorage : simple et persistant

```
// enregistrer (au login)
localStorage.setItem('token', data.token)

// lire (plus tard)
const token = localStorage.getItem('token')

// supprimer (au logout)
localStorage.removeItem('token')
```

TSX

localStorage

- Stockage cle -> valeur
- dans le navigateur
-
- Survit au rechargement
- et a la fermeture
-
- Toujours des chaines
- (strings)

Nuance securite. localStorage est parfait pour APPRENDRE et pour un projet de cours. En production, il est sensible aux attaques XSS ; les apps serieuses preferent souvent un cookie httpOnly. Pour ce cours : localStorage, c'est tres bien - mais sachez que la question existe.

Envoyer le token à chaque requête

Le header Authorization: Bearer

Méthode 1 - manuelle (au cas par cas) :

```
const token = localStorage.getItem('token')
await axios.get('http://localhost:3000/mes-donnees', {
  headers: { Authorization: `Bearer ${token}` },
})
```

TSX

Méthode 2 - une instance axios (recommandé) :

```
// src/api.ts
export const api = axios.create({ baseURL: 'http://localhost:3000' })

// on ajoute le token AUTOMATIQUEMENT à chaque requête (intercepteur)
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})
```

TSX

Avec l'intercepteur, on utilise 'api.get(...)' partout et le token est ajouté tout seul. Fini de le recopier à chaque appel.

PARTIE 3

AuthContext : l'état global de connexion

Pourquoi un contexte d'auth ?

Tout le monde doit savoir si on est connecté

La navbar veut afficher 'Se deconnecter', une page veut le profil, un bouton veut se cacher si non connecte... Beaucoup de composants ont besoin de l'etat de connexion. On le met dans un CONTEXTE (useContext, vu en S8) plutot que de le faire descendre par props.

Ce que le contexte contient

- Le token (ou null)
- Est-on connecte ? (booleen)
- seConnecter(token)
- seDeconnecter()
- -> accessible partout

Qui l'utilise

- La navbar (afficher login/logout)
- La page de connexion
- Les boutons proteges
- Les routes privees
- -> une seule source de verite

Rappel du trio (S8) : createContext (declarer) -> Provider (fournir, en haut de l'app) -> useContext (consommer, n'importe ou en dessous).

Créer le AuthContext

Un contexte + un Provider qui gère le token

TSX

```
// src/AuthContext.tsx
import { createContext, useContext, useState } from 'react'

const AuthContext = createContext<any>(null)

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [token, setToken] = useState<string | null>(
    localStorage.getItem('token') // on relit au démarrage
  )

  function seConnecter(t: string) { localStorage.setItem('token', t); setToken(t) }
  function seDeconnecter() { localStorage.removeItem('token'); setToken(null) }

  return (
    <AuthContext.Provider value={{ token, estConnecte: !!token, seConnecter, seDeconnecter }}>
      {children}
    </AuthContext.Provider>
  )
}

export const useAuth = () => useContext(AuthContext) // hook pratique
```

Brancher et utiliser le contexte

Provider en haut, useAuth partout

1) Envelopper l'app (main.tsx) :

```
<AuthProvider>  
  <App />  
</AuthProvider>
```

TSX

2) L'utiliser dans un composant :

```
const { estConnecte, seDeconnecter }  
  = useAuth()  
  
{estConnecte  
  ? <button onClick={seDeconnecter}>Logout</button>  
  : <a href='/login'>Se connecter</a>}
```

TSX

Dans le formulaire de login

- Au succes de `axios.post('/auth/login')`, on appelle `seConnecter(data.token)`
- -> le contexte met a jour le token ET `localStorage`
- -> TOUS les composants qui lisent `useAuth` se re-affichent (navbar, boutons...)
- -> on peut rediriger l'utilisateur vers la page d'accueil

PARTIE 4

Affichage conditionnel & routes protégées

Afficher selon la connexion

Montrer/cacher des éléments d'interface

```
const { estConnecte } = useAuth()

return (
  <div>
    {estConnecte && <button>Capturer</button>}

    {estConnecte
      ? <p>Bienvenue, dresseur !</p>
      : <p>Connectez-vous pour capturer.</p>}
  </div>
)
```

TSX

Deux techniques

- condition && <JSX>
- -> affiche SI vrai
-
- condition
- ? <A/>
- :
- -> A ou B
-
- Pratique pour navbar,
- boutons, messages

ATTENTION : cacher un bouton n'est PAS une securite. Un utilisateur malin peut quand meme appeler l'API. La vraie protection reste COTE BACKEND (le JWT de la seance 6). L'affichage conditionnel, c'est du confort d'interface.

Une route protégée (idée)

Rediriger si non connecté

```
// composant garde-barriere
function Protege({ children }: any) {
  const { estConnecte } = useAuth()
  if (!estConnecte) {
    return <p>Acces reserve. Connectez-vous.</p>
    // (avec react-router : <Navigate to='/login' />)
  }
  return children
}

// usage : <Protege><PageProfil /></Protege>
```

TSX

L'idée

- Un composant qui 'garde'
- une partie de l'app
-
- Si non connecte :
- message ou redirection
-
- Si connecte :
- on affiche le contenu
-
- (le routing viendra
- avec react-router)

Meme logique cote token expire : si l'API renvoie 401, on peut appeler seDeconnecter() et renvoyer l'utilisateur vers la page de connexion (via un intercepteur de reponse axios).

La déconnexion

Vider le token, revenir à l'état déconnecté

```
function seDeconnecter() {  
  localStorage.removeItem('token')  
  setToken(null)    // -> l'app repasse en 'non connecté'  
}
```

TSX

Effet immédiat

- token supprime
- estConnecté devient false
- la navbar remontre 'Login'
- les boutons protégés
- disparaissent

Le cycle complet, du clic 'Se connecter' au 'Se deconnecter' :

- **Login** : `axios.post('/auth/login')` -> `seConnecter(token)` -> stocke + met à jour l'état
- **Navigation** : l'intercepteur ajoute le token à chaque requête protégée
- **Affichage** : `useAuth()` décide quoi montrer (login vs logout, boutons)
- **Logout** : `seDeconnecter()` -> vide le token -> retour à l'état initial

PARTIE 5

Exercice : login sur le Pokédex

Exercice : sécuriser le front Pokédex

Ajouter la connexion à votre app React

- 1. **Créer** un composant Login (formulaire email + mot de passe)
- 2. **AuthContext** avec token, estConnecte, seConnecter, seDeconnecter
- 3. **Envelopper** l'app dans <AuthProvider> (main.tsx)
- 4. **Instance axios** avec intercepteur qui ajoute le token
- 5. **Afficher** 'Se connecter' ou 'Se deconnecter' selon l'etat
- 6. **Protéger** le bouton 'Capturer' : visible seulement si connecte

Rappel : vos routes /auth/register et /auth/login existent déjà (seance 6). On les BRANCHE au front.

Détail pas à pas dans le guide : [EXERCICE-AUTH-FRONT-POKEDEX.md](#)

Récapitulatif de la séance

Ce qu'il faut retenir

Le flux

- login -> token
- stocker (localStorage)
- envoyer (Authorization)
- logout -> vider
- backend = vraie securite

Les outils

- axios.post('/auth/login')
- localStorage set/get/remove
- intercepteur axios
- AuthContext + useAuth
- affichage conditionnel

À retenir

- cacher != securiser
- la securite est au back
- un contexte = 1 source
- de verite pour l'auth
- 401 -> se deconnecter

L'essentiel en code :

```
localStorage.setItem('token', data.token) // au login
config.headers.Authorization = `Bearer ${token}` // intercepteur
const { estConnecte, seDeconnecter } = useAuth() // partout
```

TSX

Pour la prochaine séance

On consolide et on prépare la suite

À faire de votre côté

- Ajouter Login au front Pokedex
- Créer le AuthContext
- Intercepteur axios (token)
- Cacher 'Capturer' si non connecte
- Bouton de deconnexion
- Pousser sur GitHub

Au programme ensuite

- react-router (vraies pages)
- Routes privées + redirections
- Refresh token / expiration
- Formulaires avancés (validation)
- Vers Next.js (full-stack)
- Déploiement (Vercel + Render)

La boucle est bouclée : login au back, login au front.

Votre application gère de vrais utilisateurs connectés, du navigateur à la base.



Questions ?

Prochaine étape : connecter les utilisateurs dans votre Pokédex.

Login, token, contexte. Le full-stack, complet.

